

IF 260

Sistem Operasi

Minggu 2

Dr. Ananda Kusuma
e-mail: ananda.kusuma@lecturer.umn.ac.id

Universitas Multimedia Nusantara
Serpong, Tangerang

Agenda

- **Review**
- **Topik Minggu 2:**
 - **Hardware:**
 - Processor, memory, magnetic disk & tape
 - **Konsep-konsep abstraksi pada sistem operasi**
 - **System calls**
 - **Struktur sistem operasi**

Review

Peranan Sistem Operasi

- Sistem operasi adalah suatu software yang bertindak sebagai perantara antara pengguna/aplikasi dengan sumber daya hardware komputer. Peranannya dapat ditinjau sbb:
- Top-view: Extended Machine (**high-level abstraction**)
 - Menyembunyikan kerumitan/instruksi detil dari sumber daya hardware → software lebih portable
 - Penggunaan abstract object seperti process/thread, files, virtual memory, socket. Sistem operasi menyediakan system call untuk pengguna
- Bottom-up: **Resource Manager**
 - Multiplexing/sharing sumber daya
 - In time: penggunaan sumber daya bersamaan (concurrency)
 - In space: alokasi pada memory dan disk
 - Multi-user, di mana sumber daya (memory, disk, I/O devices, dsb) untuk tiap pengguna diatur dan dilindungi

Hardware (Perangkat Keras):

- Processor**
- Memory**
- Disk**
- I/O devices**

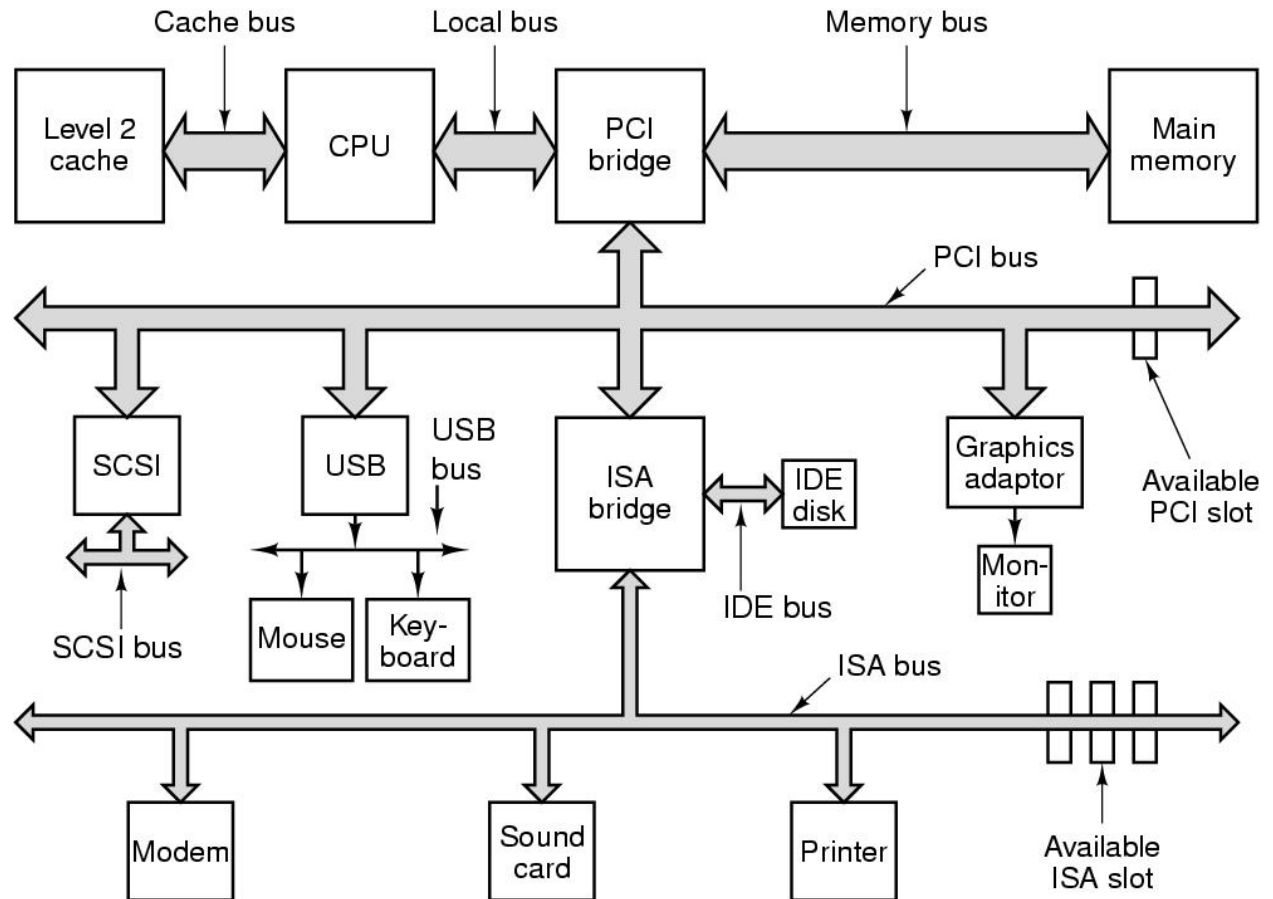
Computer Hardware

- Processors
- Memory
- Disks
- Tapes
- I/O Devices
- Buses



Buses (interkoneksi antar komponen) pada Sistem PC Pentium

Cache:
L1: internal (8Kb-128Kb)
L2: external (256Kb-2Mb)
L3: untuk multi-core
(kontrol data dari L2 cache
di tiap core



Processor (1)

- **Processor/CPU (Central Processing Unit)**
 - Otak dari sistem komputer. Mengambil instruksi dari memory, decode instruksi, ambil data yang diperlukan, dan eksekusi. Perkembangan terkini → CPU multicores yang memiliki beberapa processors dalam satu chip
 - Spesifik instruksi untuk CPU berbeda, mis. Intel Pentium, Sun SPARC, Motorola, PowerPC, dsb.
 - Terdiri dari CU (Control Unit) , ALU (Aritmetic Logic Unit) dan memiliki registers:
 - Register yang terlihat oleh pemakai
 - Register data (Accumulator):
 - general purpose
 - special purpose
 - Register alamat: berisi alamat data dan instruksi di memory utama, mis. index register, segment pointer register, stack pointer register.
 - Register untuk kendali/status (control & status registers)

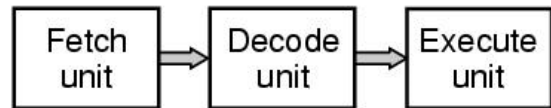
Kenapa CPU memerlukan registers?

Processor (2)

-
- **Register untuk kendali/status:** untuk pengendalian processor operation. Sebagian dapat diakses dengan instruksi yang dieksekusi di mode kernel
 - **Register untuk alamat dan buffer**
 - Mis. MAR (memory address register), IO AR (I/O address register)
 - **Register untuk eksekusi instruksi**
 - PC (Program Counter) → mencatat alamat memory dari instruksi yang akan diambil untuk dieksekusi
 - IR (Instruction Register) → menampung instruksi yang telah diambil (fetch)
 - **Register untuk informasi status/PSW (program status word)**
 - Status: sign, zero, equal, overflow, interrupt enable/disable, supervisor, dsb.

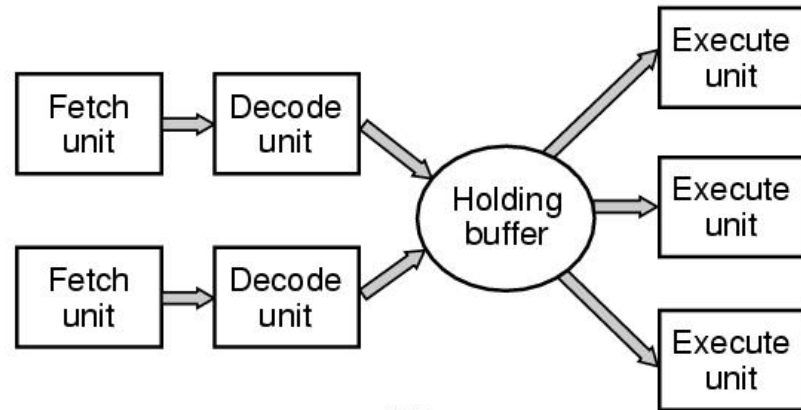
Processor (3)

- Model eksekusi sederhana → fetch, decode dan execute satu instruksi pada satu waktu
- Untuk meningkatkan kinerja CPU, dapat dilakukan teknik pipeline, superscalar, hyperthreading/multithreading
→ eksploitasi **parallelisme** dalam pemrosesan



(a)

(a) A three-stage pipeline.



(b)

(b) A superscalar CPU.

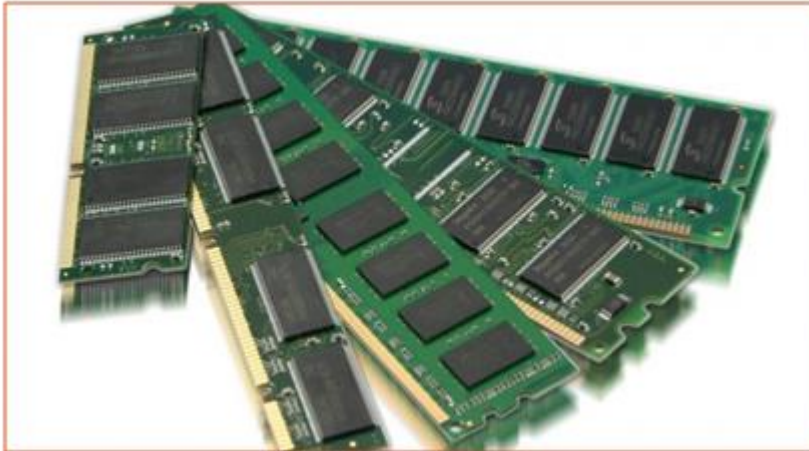
- Arsitektur superscalar memiliki beberapa *functional units*
- Implikasi instruksi-instruksi dapat dieksekusi secara tidak berurutan

Multithreading / Hyperthreading

- **CPU dapat menyimpan state dari beberapa thread berbeda, dan bolak balik menjalankannya pada skala waktu nanosecond.**
 - **Contoh: jika salah satu thread reads a word from memory (memerlukan beberapa clock cycles), maka multithread CPU bisa switch ke thread lainya untuk mengerjakan hal yang lain.**
 - **Catatan: pada satu waktu tetap satu process yang berjalan, namun thread switching sangat cepat (orde nanosecond)**
- **Untuk sistem operasi, tiap thread ini akan terlihat sebagai CPU yang berbeda**
 - **Contoh: sistem dengan 2 CPUs, dan masing-masing menjalankan 2 threads, oleh sistem operasi akan terrihat sebagai sistem yang memiliki 4 CPUs**

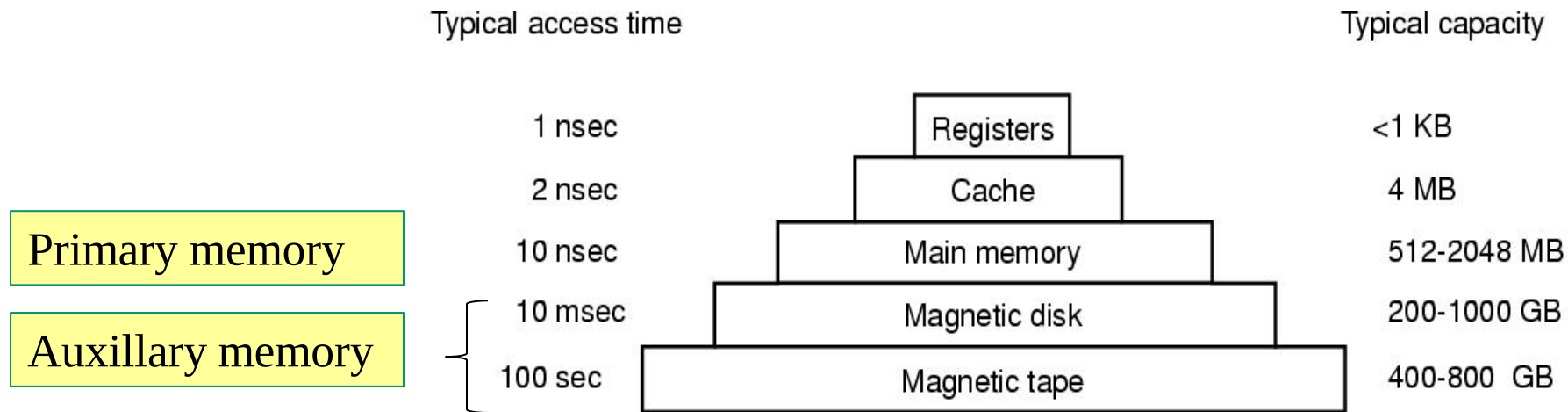
Memory (1)

- Berfungsi menyimpan program dan data
- Idealnya pengaksesan memory (*memory reference*) berjalan cepat, sehingga kerja CPU tidak tertahan oleh memory.



Memory (2)

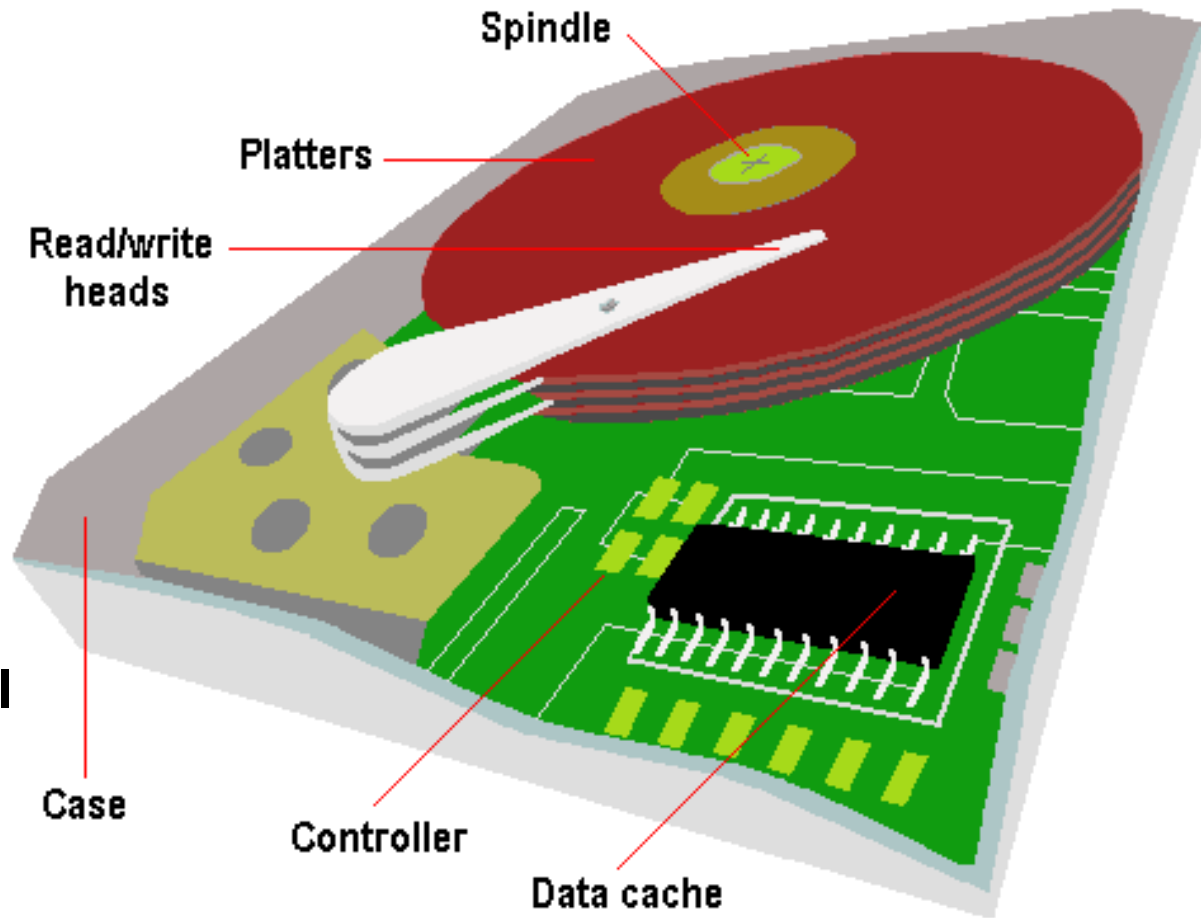
- Trade-off: capacity, access time, cost → memory hierarchy



– Main/Primary memory

- RAM (Random Access Memory) → volatile
- ROM (Read Only Memory) → nonvolatile dan berisikan bootstrap
- EEPROM (Electrically Erasable PROM) dan flash memory
→ nonvolatile dan data dapat dihapus atau ditulis ulang
- CMOS (Complementary metal-oxide semiconductor) --> untuk menyimpan tanggal, waktu dan konfigurasi BIOS. Daya rendah, dan cukup menggunakan baterai untuk mempertahankan data

Magnetic Disk

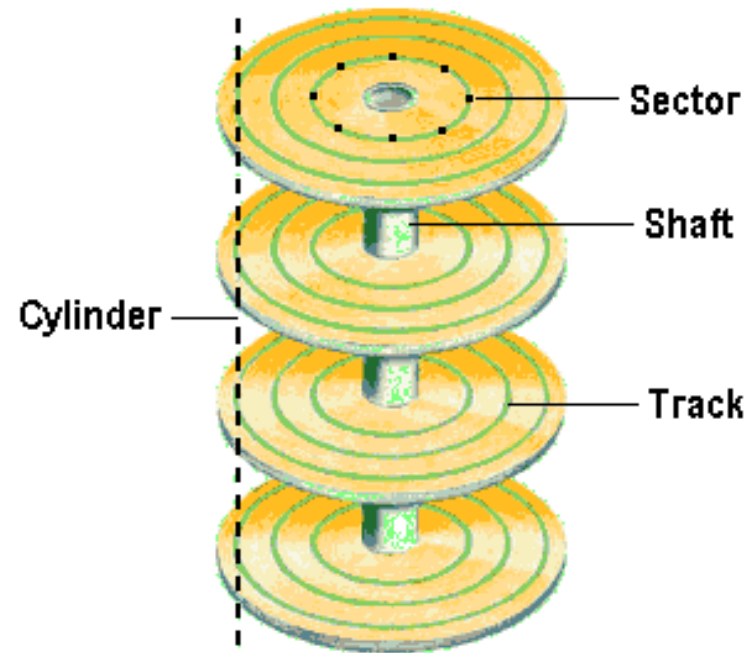


Standard Koneksi external

- IDE/ATA/SATA
- SCSI
- USB

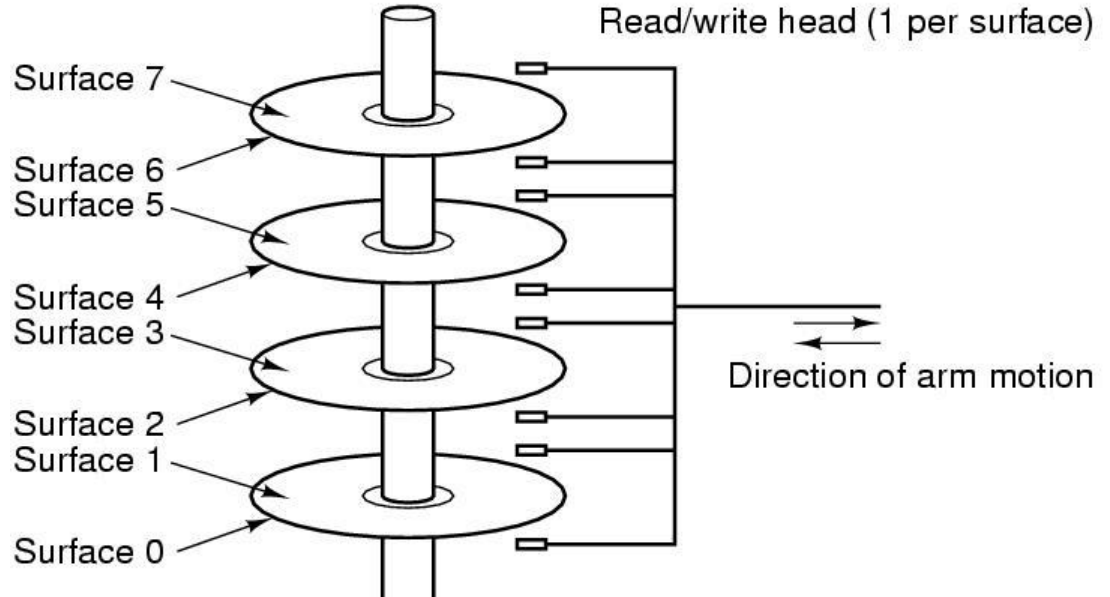
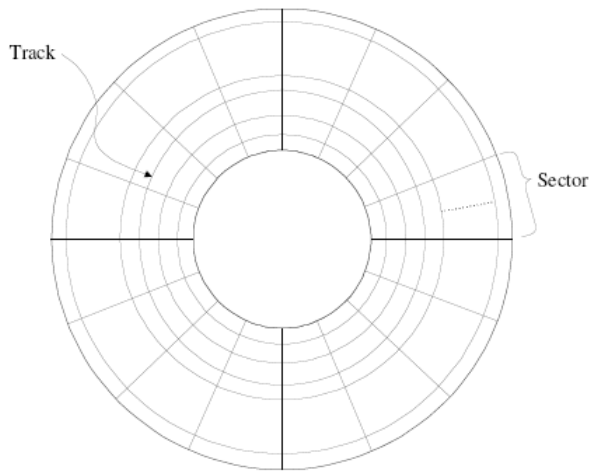
Harddisk Geometri

- **Platters**
 - Lempengan material magnetic 2 sisi
 - 2 heads untuk read/write
 - Kecepatan putar 3000-15000 RPM
- **Tracks**
 - Lingkaran concentric
 - Dibagi atas beberapa sectors
- **Cylinders**
 - Track yang sama pada tiap plater
- **Operasi**
 - Seek: gerakkan arm ke track
 - Read/write: tunggu sampai sector berada di bawah read/write head, kemudian transfer data
- Controller konversikan physical address [cylinder, head, sector] menjadi logical disk address berupa array of sectors atau blocks(group of sector)
- [cylinder,head,sector] → CHS addressing



Magnetic Disk (Hard Disk)

- Akses lambat dibandingkan RAM karena disk merupakan perangkat mekanis. Waktu transfer data meliputi
 - seek time (move the read/write heads to the appropriate cylinder)
 - rotational latency (wait until the desired sectors spin the read/write heads)
 - transfer time



Magnetic Tape

- This medium is often used as a backup for disk storage and for holding very large data sets.
- The big plus of tape is that it is exceedingly cheap per bit and removable, which is important for backup tapes that must be stored off-site in order to survive fires, floods, earthquakes, and other disasters.

Karakteristik:

Pengaksesan data secara **sequential**



Booting Process

- Setelah komputer dihidupkan dan mengaktifkan seluruh komponen pada motherboard, maka program yang pertama kali dieksekusi adalah BIOS yang berada di chip ROM (atau flash RAM)
- BIOS berisikan instruksi untuk melakukan POST (Power-on self-test) untuk memeriksa apakah semua komponen dan peripheral berfungsi. Apabila semua berfungsi, kemudian BIOS mencari boot device dari informasi konfigurasi di memory CMOS
- Program yang ada di sektor pertama pada boot device dibawa ke memory dan dieksekusi. Program ini kemudian membaca partition table untuk melihat partisi yang aktif dan bootloader yang tersedia untuk load sistem operasi
- Sistem operasi mengendalikan komputer. Kernel resident selama komputer beroperasi. Apabila diperlukan OS load program-program (non-resident) lainnya yang diperlukan
- OS kemudian menampilkan login prompt atau GUI

Kernel

- **Bagian inti dari suatu sistem operasi, yang memiliki kendali sepenuhnya atas sumber daya hardware**
 - **Processor status: supervisor atau privileged**
- **Kernel code diletakkan pada area memory yang terlindungi (protected) sehingga tidak bisa ditimpa oleh program-program yang lain. Area memory ini disebut kernel address space**
- **Di-loaded ke memory saat booting dan tetap berada di sana selama komputer digunakan (lihat materi kuliah minggu lalu tentang proses booting)**
- **Sistem crash/freeze, artinya kernel crash/freeze**
- **Fungsi-fungsi sistem operasi apa yang diletakkan pada kernel?**
 - **topik tentang struktur sistem operasi yang akan dijelaskan pada sesi terakhir**

Konsep-konsep Abstraksi pada Sistem Operasi

Abstraksi (high-level abstraction)

Hardware	Contoh OS Services	Abstraksi untuk user
Processor	Process management, Scheduling, Traps, Protections, Synchronization	Process
Memory	Management, Protection, Virtual memory	Address space
I/O devices	Concurrency with CPU, Interrupt handling	Terminal, Mouse, Printer
File system	Management, Persistence	Files
Distributed systems	Network security, Distributed file system	Sockets, RPC system calls, Transparent file sharing

Dukungan hardware untuk OS Services

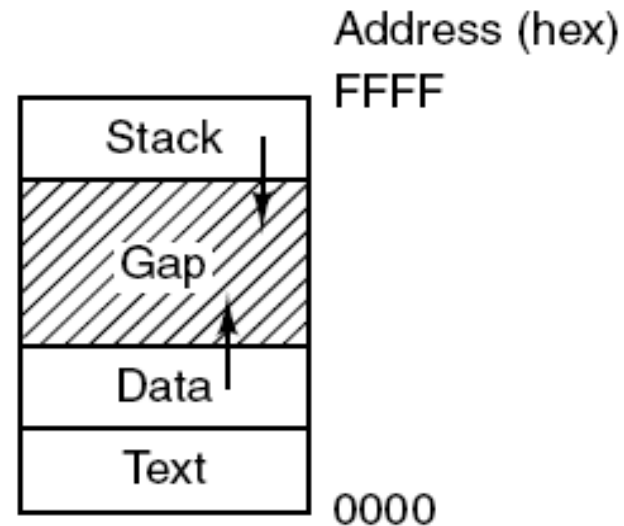
OS Service	Hardware Support
Protection	Kernel / User mode Protected Instructions Base and Limit Registers
Interrupts	Interrupt Vectors
System calls	Trap instructions and trap vectors
I/O	Interrupts or Memory-Mapping
Scheduling, error recovery, billing	Timer
Synchronization	Atomic instructions
Virtual Memory	Translation look-aside buffers Register pointing to base of page table

Proses (Process)

- **Process adalah**
 - Program yang sedang dieksekusi (berjalan)
 - Container (wadah) yang berisikan seluruh informasi yang diperlukan untuk menjalankan program
 - Program : static file (image), misalnya executable file atau library file
 - Process : executing program = program + execution state
- **Tiap process memiliki PID (Process ID)**
- **Program yang sama dapat dieksekusi beberapa kali, dan tiap eksekusi diabstrasikan dengan process yang berbeda.**
- **Process dapat menduplikasi (clone) process nya**
 - Command `fork()` in Unix untuk membuat child process

Sumber daya (Process Resources) (1)

- **Tiap process memiliki**
 - **Lokasi pengalamatan atau Address Space (core image) yang meliputi 3 segments: text (program code), (variables/heaps), dan stack**



Contoh Ilustrasi

Source code



Process di address space

```
double r;  
char c;  
...  
void Y (int b) {  
    if (b == 1) {  
...  
int main() {  
    int a = 2;  
    Y(a);  
}
```

PC
→

SP →

HP →

PC →

main; a = 2

Y; b = 2

Stack



r,c

Heap

```
void Y (int b) {  
    if (b == 1) {  
...  
int main() {  
    int a = 2;  
    Y(a);  
}
```

Code

PC : Program Counter

SP : Stack Pointer

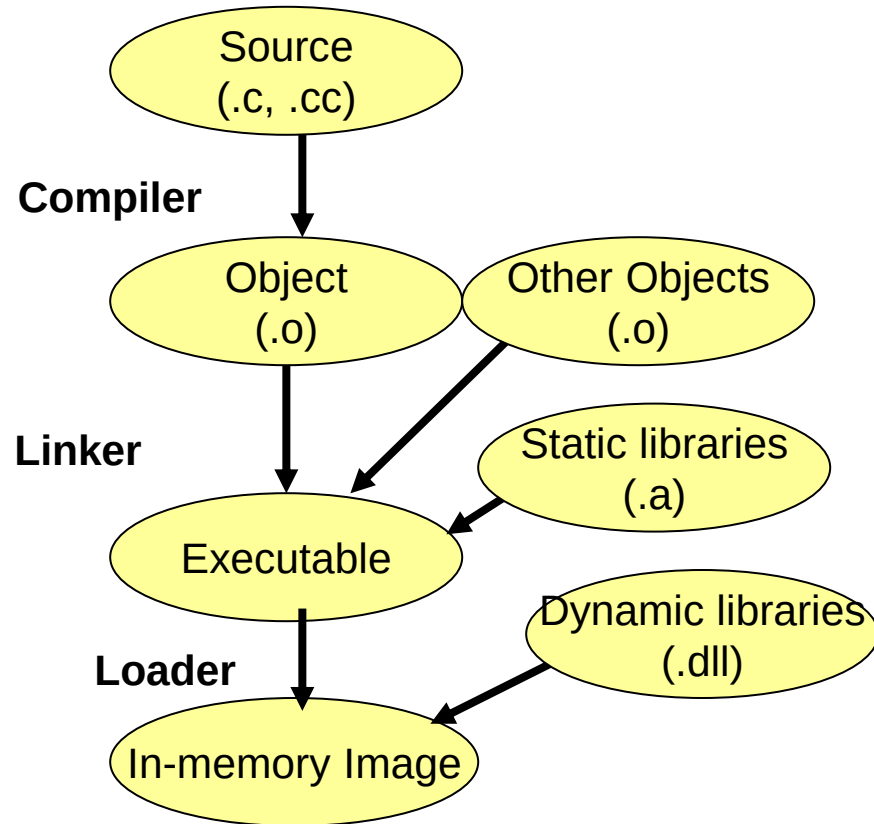
HP : Heap Pointer

Sumber daya (Process Resources) (2)

- **Tiap process juga memiliki**
 - **Atribut/informasi pada process table atau process control block (PCB) yang berisikan berbagai sumber daya spt. Informasi registers (program counter, stack pointer), open files, alarm, process-process terkait, dsb.**

Process management	Memory management	File management
Registers Program counter Program status word Stack pointer Process state Priority Scheduling parameters Process ID Parent process Process group Signals Time when process started CPU time used Children's CPU time Time of next alarm	Pointer to text segment info Pointer to data segment info Pointer to stack segment info	Root directory Working directory File descriptors User ID Group ID

Dari Source Codes menjadi Process



Untuk menjadikan process:

Loader (bagian dari OS) membaca executable file, kemudian memetakan address di dalam file ke memory address, dan menghubungkan ke dynamic library yang digunakan

Contoh:

- Login dan buka shell
- Dengan file editor, buat file hello.c sbb.:

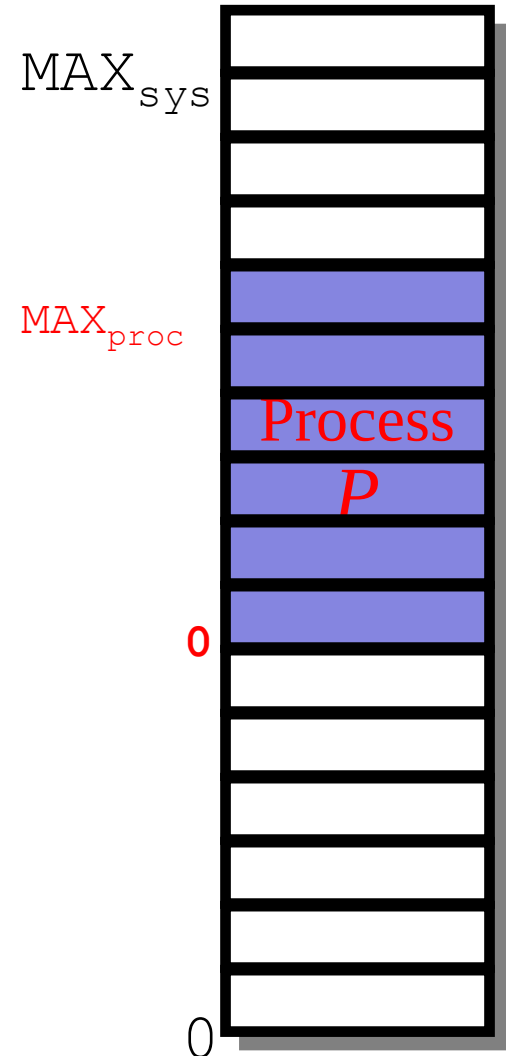
```
#include <stdio.h>
main()
{
    while(1)
        printf("Hello world\n");
}
```
- Buat executable file: `$ gcc -o hello hello.c`
- Jalankan executable file: `$ ./hello`
- Amati process dengan: `$ top`
- Jalankan lagi hello pada shell yang berbeda dan amati keluaran dari top

Memory Management

- **Multiple programs dalam memory memerlukan:**
 - Pengaturan lokasi
 - Proteksi
- **OS membuat abstraksi memory (memory abstraction) pada media penyimpanan (storage media) dan mengelolanya (manage)**
- **Abstraksi memory → address space**
 - Physical address space: address space yang disediakan hardware
 - Logical address space: address space yang dilihat oleh process

Address Space (Ruang Alamat)

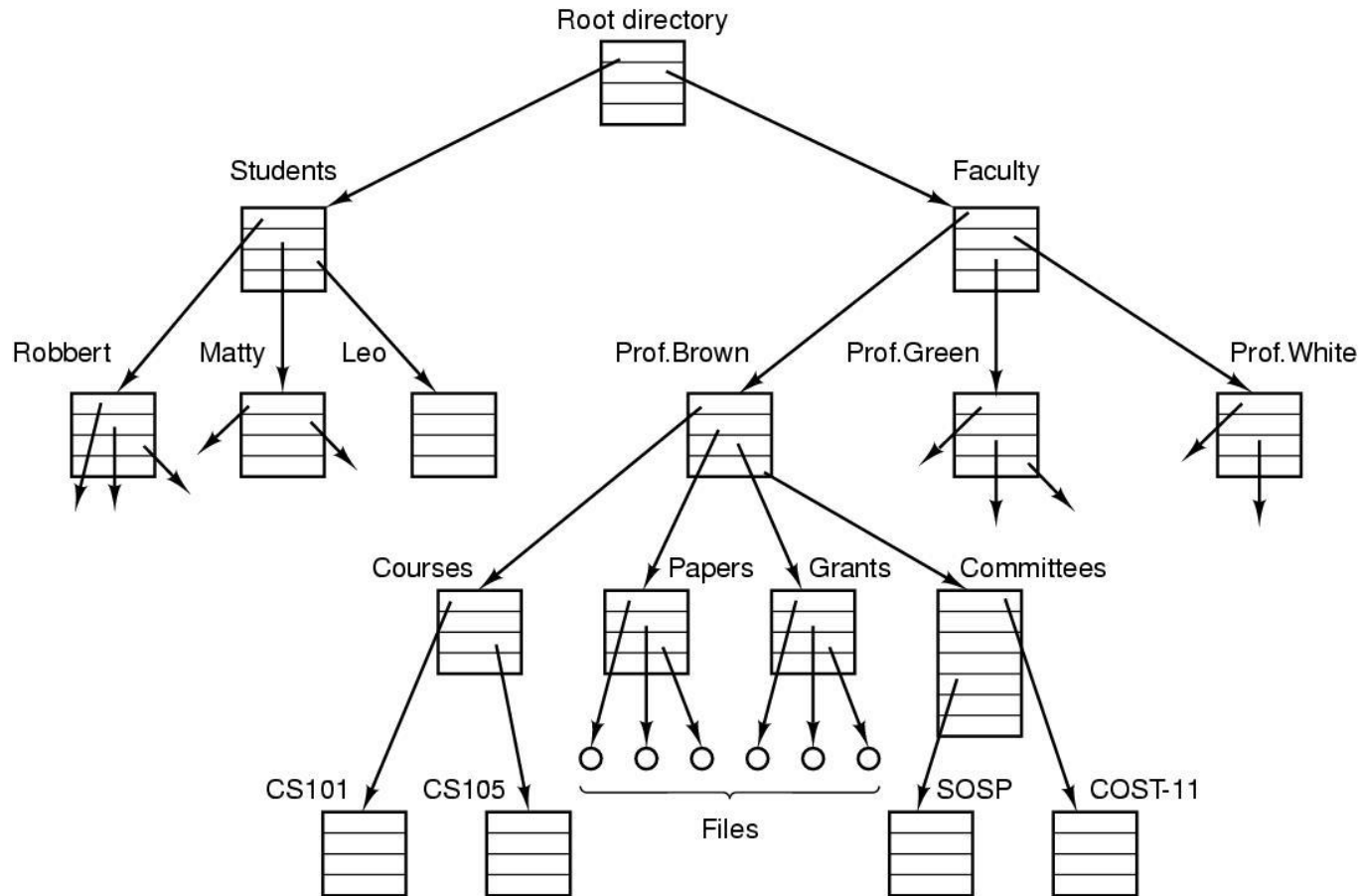
- **Physical address space (ruang alamat fisik):**
 - address space yang disediakan oleh hardware (RAM), contoh: $0\text{-}MAX_{sys}$
- **Logical address space (ruang alamat logis):**
 - address space yang dilihat oleh process, contoh: $0\text{-}Max_{proc}$
- **Tugas OS sebagai memory manager di antaranya**
 - Memetakan logical address ke physical address
 - Mengatur penempatan process ke physical memory untuk meminimalkan fragmentation
 - Mengelola virtual memory yang memungkinkan program untuk berjalan walau hanya secara parsial berada di main memory



File System (Sistem Berkas)

- **File** adalah abstraksi dari kumpulan informasi yang berhubungan dan tersimpan dalam secondary storage
 - Sifatnya persistence, di mana file hanya akan hilang apabila owner (pemiliknya) secara eksplisit menghapusnya
 - Memiliki: file naming, file structure, file type, file access, file attributes
- **Directory/folder** adalah system file sebagai
 - abstraksi untuk mengelompokkan beberapa files untuk menyusun struktur dari file system
- **Special files** → I/O devices dapat dimodelkan sebagai file sehingga system call yang sama untuk read dan write dapat digunakan
 - Block special files: randomly addressable blocks
 - Character special files: character streams
- **OS** bertugas untuk menentukan file structure, file naming, file attributes, mengelola (membuat (create), menghapus (remove), membaca (read), menulis (write) files, dsb.

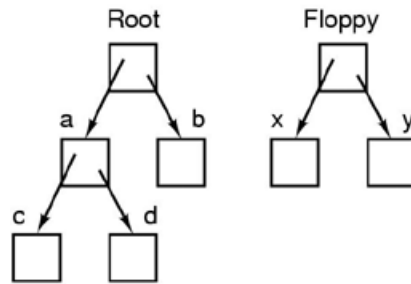
Directory Hierarchy



Konsep:

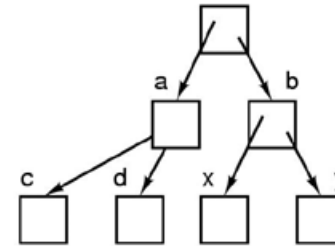
- Root directory, home directory, working directory
- Absolute path, relative path

Mounting, Linking dan Pipe



(a)

Sebelum mounting



(b)

Setelah mounting

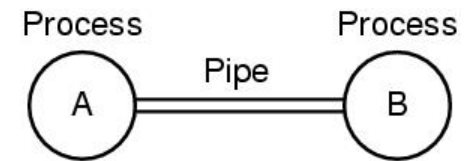
/usr/ast			/usr/jim		
16	mail		31	bin	
81	games		70	memo	
40	test		59	f.c.	
			38	prog1	

(a)

/usr/ast			/usr/jim		
16	mail		31	bin	
81	games		70	memo	
40	test		59	f.c.	
70	note		38	prog1	

(b)

system call untuk linking
`link("/usr/jim/memo", "/usr/ast/note")`



Standard output dari A menjadi
 standard input dari process B
 contoh:

`man ls | grep "author"`

System Calls

System Call

- Suatu cara yang digunakan oleh user-level process untuk memperoleh **layanan kernel**
- **Contoh:**
 - User-level process tidak dapat secara langsung mengakses disk. Oleh karena itu, meminta kernel untuk mendapatkan data yang diperlukan dari sebuah file (read system call)
 - User level process tidak dapat membuat process yang lain. Oleh karena itu, meminta kernel untuk membuatkan (fork system call).
 - Semua services yang diimplementasikan oleh sistem operasi diakses hanya melalui system calls, missal user-level process meminta kernel melakukan sesuatu, atau via interrupts ketika device memiliki sesuatu yang perlu dilakukan oleh sistem operasi

System Call vs Procedure Call

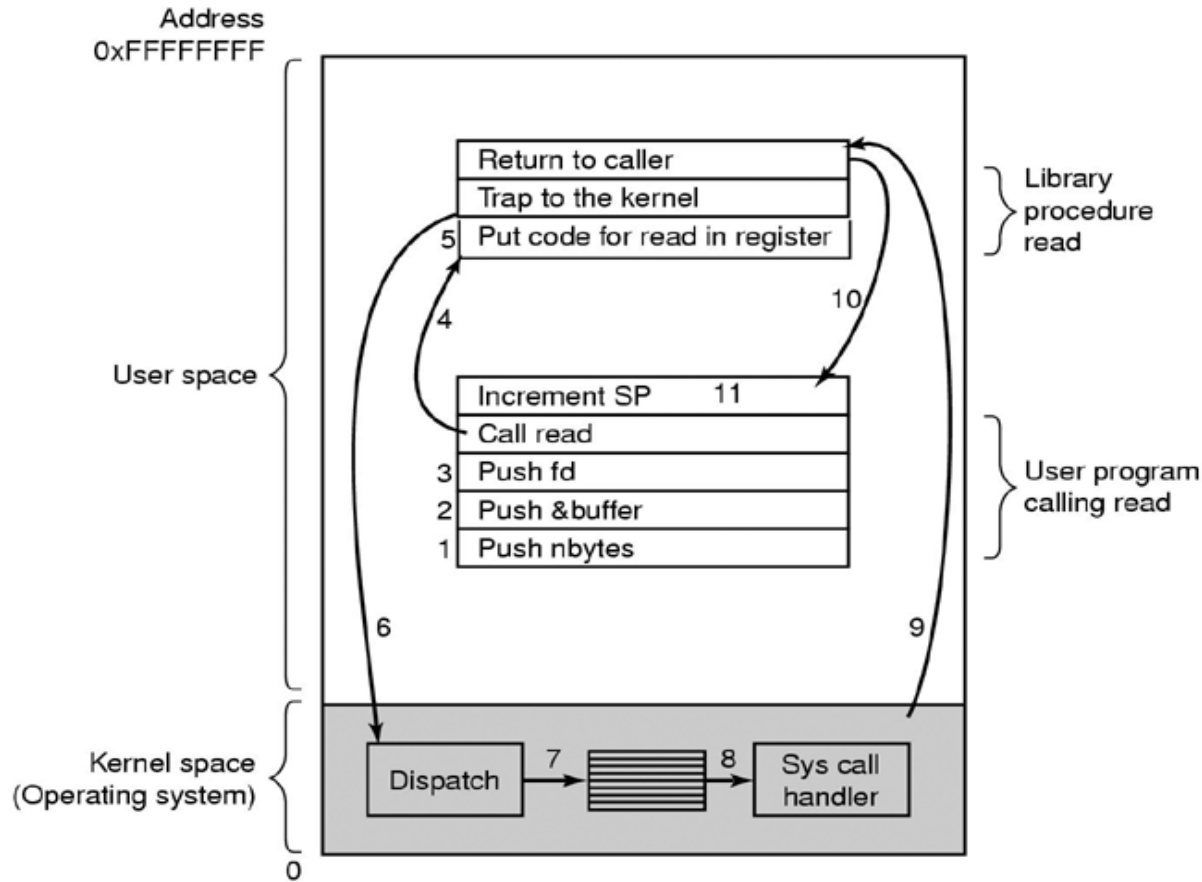
- **Penggunaan system call mirip dengan procedure call, namun system call dilakukan dengan melewati proteksi domain user-kernel, yaitu:**
 - Sebelum melakukan system call, program berjalan di user mode pada user's address space
 - Eksekusi system call dilakukan di supervisor mode pada kernel's address space
 - Supervisor mode memiliki kendali perangkat keras, dan system call interface mencegah user programs membuat sistem operasi crash
- **System calls mensyaratkan dukungan hardware, yang menyediakan **trap** instruction**

Trap Instruction

- Apabila suatu user-level process memerlukan layanan OS, maka process akan mengeksekusi suatu instruksi yang disebut trap instruction untuk memindahkan kendali ke OS.
- Trap instruction adalah software interrupt (int 80h pada arsitektur x86) yang:
 - Switch status cpu ke supervisor mode
 - Mencari address dari fungsi kernel yang hendak dijalankan dari trap table
 - Menjalan fungsi kernel tersebut
- Pertanyaan: apa beda trap (software interrupt), hardware interrupt, dan exception?

Langkah-langkah dalam membuat system call

System call dari C program untuk membaca data dari file
`count = read(fd, buffer, nbytes)`



Jelaskan dua perbedaan antara system call dan procedure call yang biasa.

Contoh POSIX System Calls

Process management

Call	Description
<code>pid = fork()</code>	Create a child process identical to the parent
<code>pid = waitpid(pid, &statloc, options)</code>	Wait for a child to terminate
<code>s = execve(name, argv, environp)</code>	Replace a process' core image
<code>exit(status)</code>	Terminate process execution and return status

File management

Call	Description
<code>fd = open(file, how, ...)</code>	Open a file for reading, writing, or both
<code>s = close(fd)</code>	Close an open file
<code>n = read(fd, buffer, nbytes)</code>	Read data from a file into a buffer
<code>n = write(fd, buffer, nbytes)</code>	Write data from a buffer into a file
<code>position = lseek(fd, offset, whence)</code>	Move the file pointer
<code>s = stat(name, &buf)</code>	Get a file's status information

Struktur Sistem Operasi

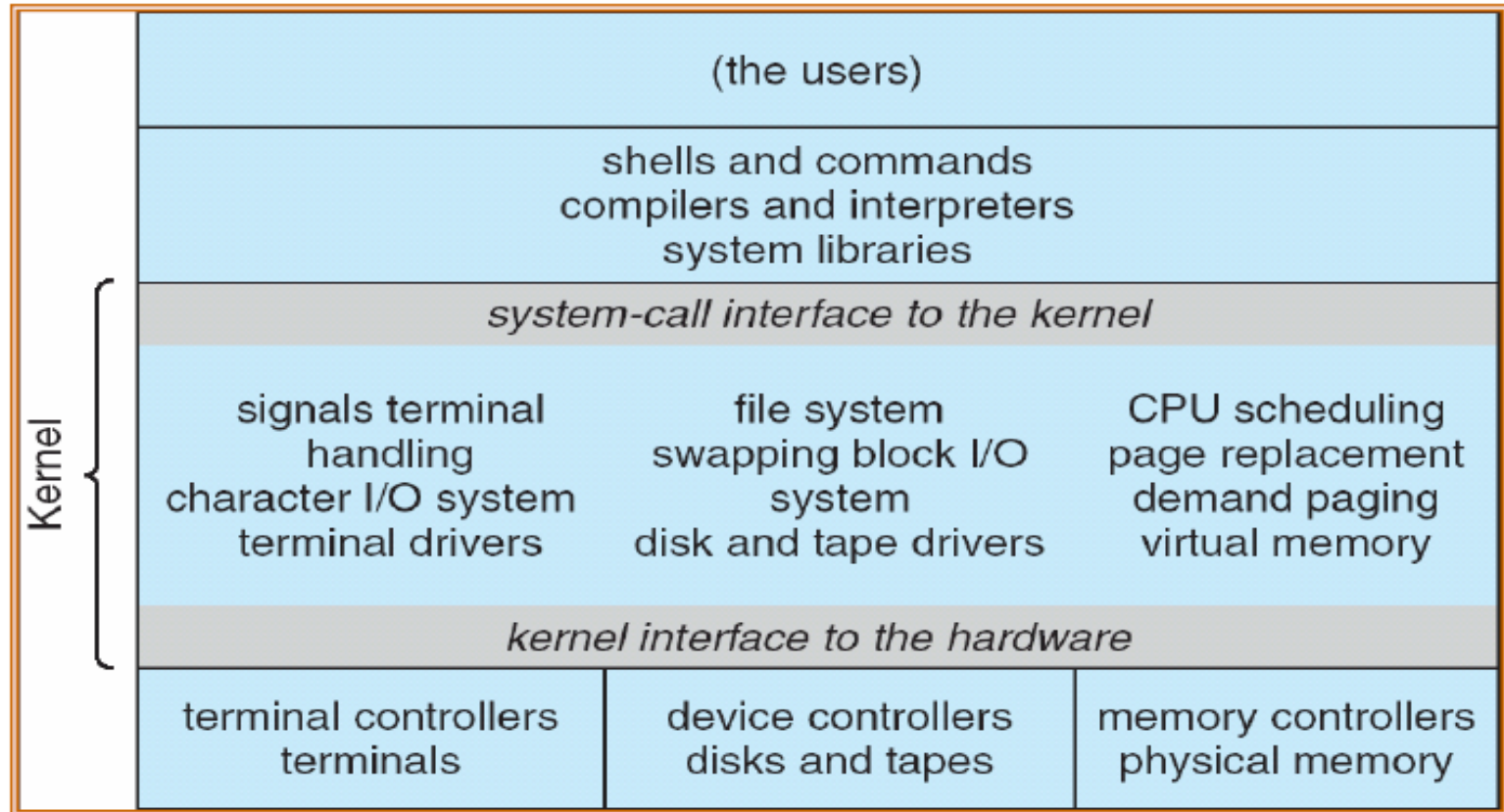
Struktur Sistem Operasi

- **Perdebatan tentang struktur sistem operasi adalah menentukan fungsionalitas mana saja yang disertakan di kernel dan bagaimana urutan (order) pemanggilan procedure nya**
 - **Kernel adalah bagian dari OS yg berjalan pada kernel mode, di mana struktur data dan device registers nya harus dilindungi dari user program**
- **Struktur sistem operasi umumnya**
 - **Monolithic**
 - **Layered**
 - **Micro-kernel**
 - **Client-server**

Monolithic

- **Struktur yang digunakan oleh OS tradisional**
- **Keseluruhan fungsi-fungsi OS dijalankan sebagai satu program di kernel**
- **Keseluruhan procedure di-linked menjadi satu executable binary program yang besar**
 - **Tiap procedure dan interrupt handler dapat memanggil satu dengan yang lain.**
- **Kelebihan: cepat dan model yang sederhana untuk coding**
- **Kekurangan: potensi unstable karena ketergantungan keseluruhan komponen (mis. bugs yang ada di satu prosedur dapat mempengaruhi OS keseluruhan)**

Contoh Monolithic: Unix OS



Modern monolithic kernels, spt. Linux, dll, memiliki fitur dengan loadable modules
→ Dapat meminimalkan ukuran kernel code, karena module code yang diperlukan (misal device driver) hanya di-load apabila diperlukan

Layered

- Menggunakan pendekatan layered, di mana ada tingkatan untuk suatu procedure memanggil procedure yang lain
 - Contoh: struktur perancangan sistem operasi THE

Layer	Function
5	The operator
4	User programs
3	Input/output management
2	Operator-process communication
1	Memory and drum management
0	Processor allocation and multiprogramming

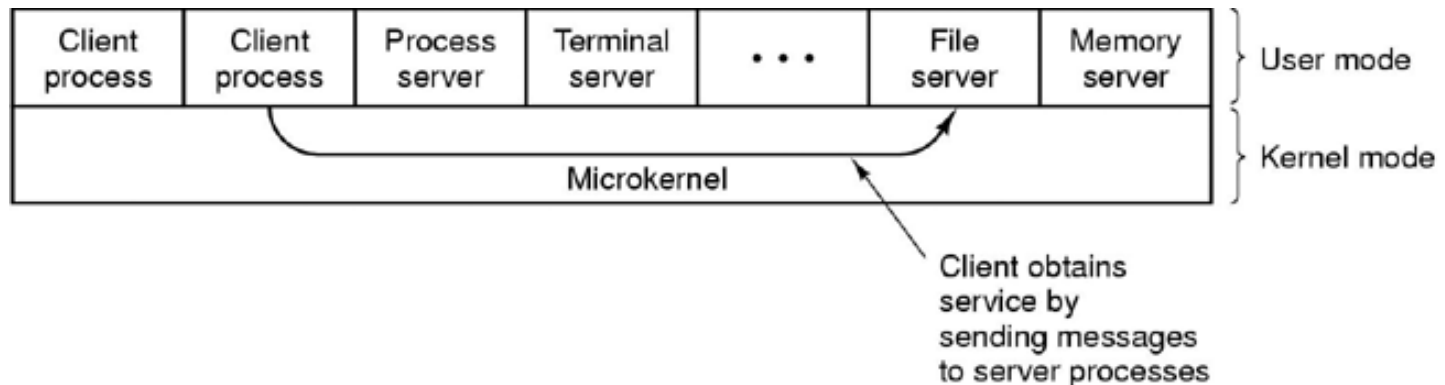
- Kelebihan: modular dan portable sehingga menyederhanakan proses design, debugging,
- Kekurangan: communication overhead antar layer, dan perlu tambahan usaha untuk copying dan book-keeping

Micro-kernel

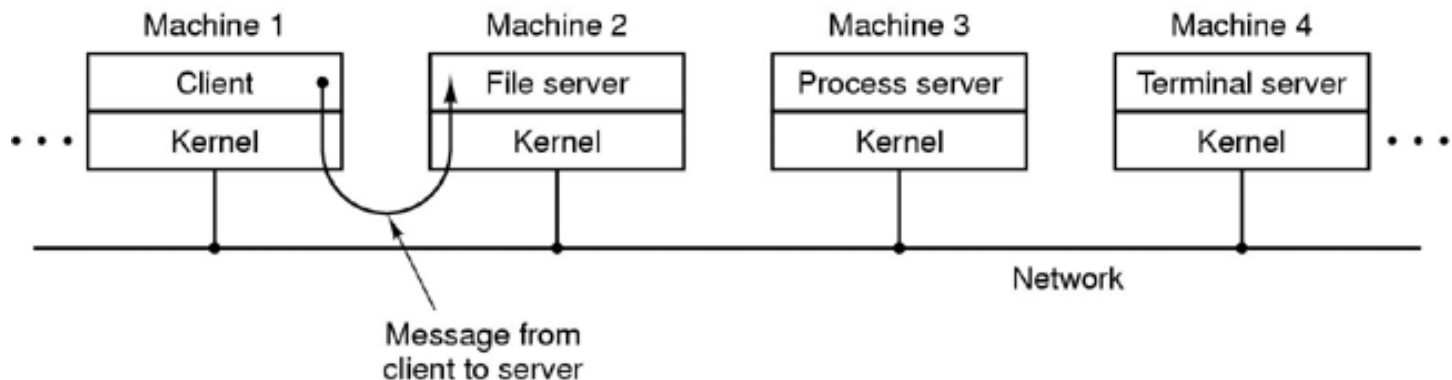
- Menggunakan pendekatan layered dan sebisa mungkin memindahkan fungsi-fungsi sistem ke user mode layer.
 - Kernel sekecil mungkin (micro-kernel), dan hanya mengatur services terkait communication dan processes → menuju client-server model
- Kelebihan: simplicity, reliability, modularity (lebih mudah dikustomisasi), dan dapat diterapkan pada distributed system
- Kekurangan: extra overhead untuk komunikasi dan sinkronisasi antar process

Client-Server Model

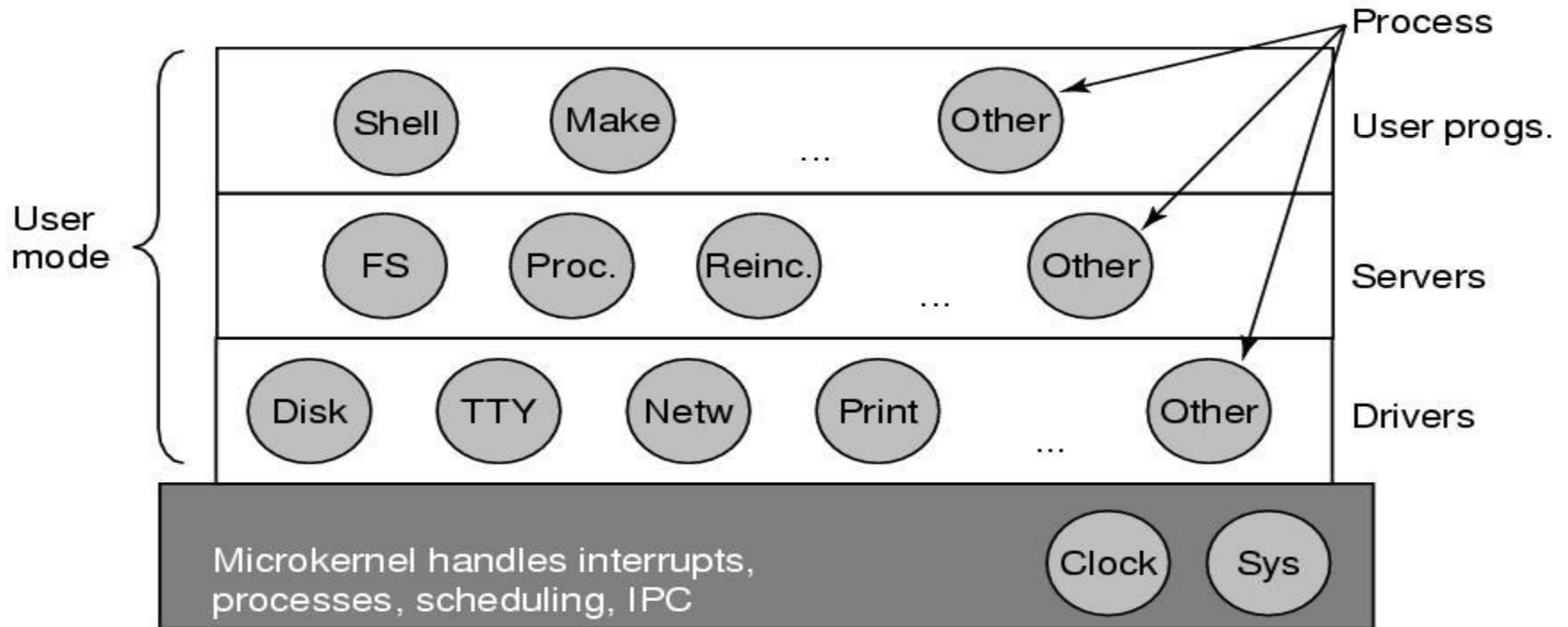
- Pengembangan dari ide micro-kernel, di mana ada dua kelas user mode processes (yg dikordinasikan oleh micro-kernel):
 - Client process: pengguna services
 - Server process: penyedia sistem services



- Ide yang sama dapat diterapkan pada jaringan komputer (distributed system)



Contoh Micro-kernel: Minix OS



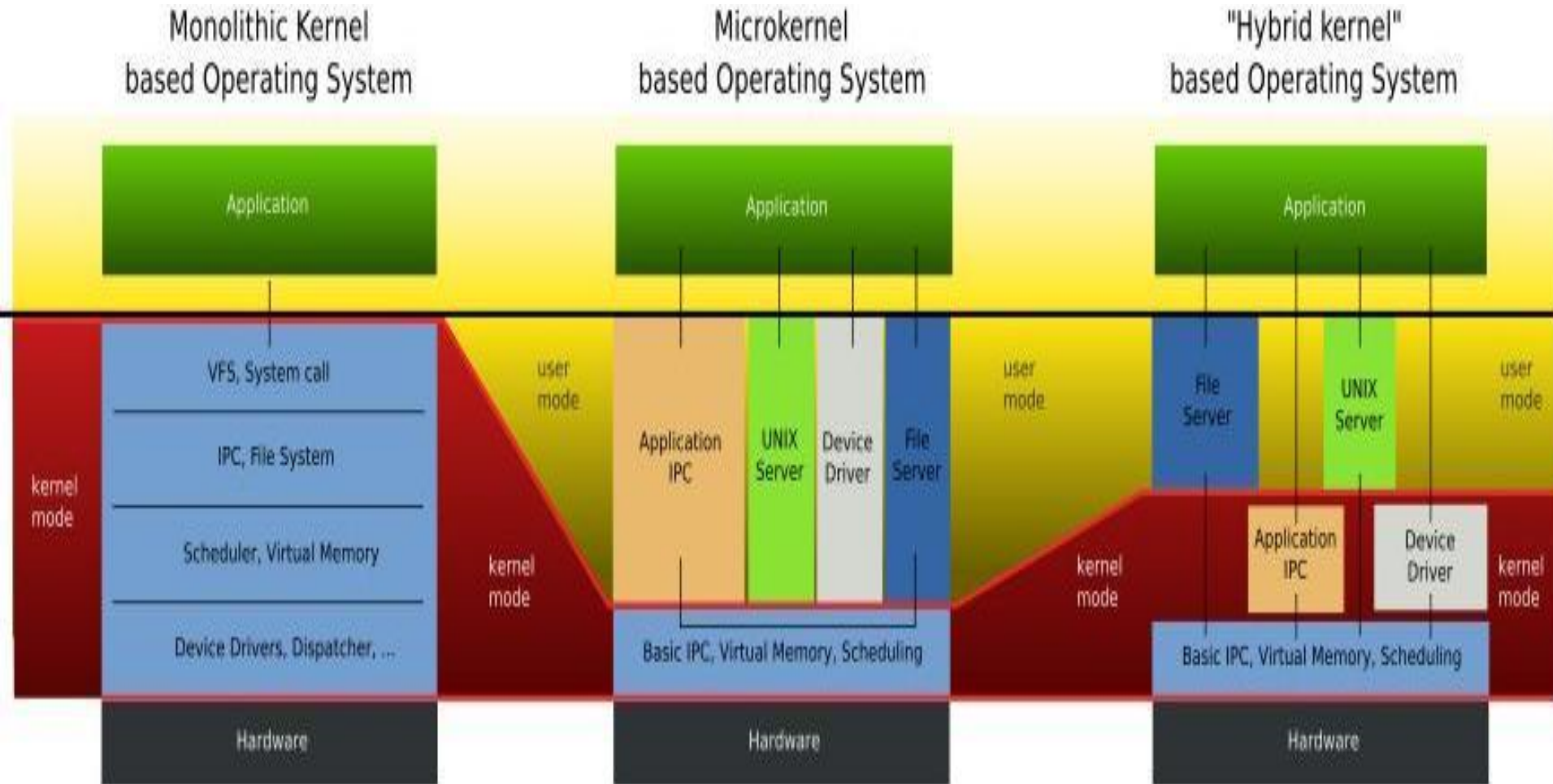
Struktur dari MINIX3 OS

Beberapa modern OS awalnya dirancang dengan semangat micro-kernel, namun untuk meningkatkan performance berbagai fitur layanan OS ditambahkan ke kernel → **Hybrid model**

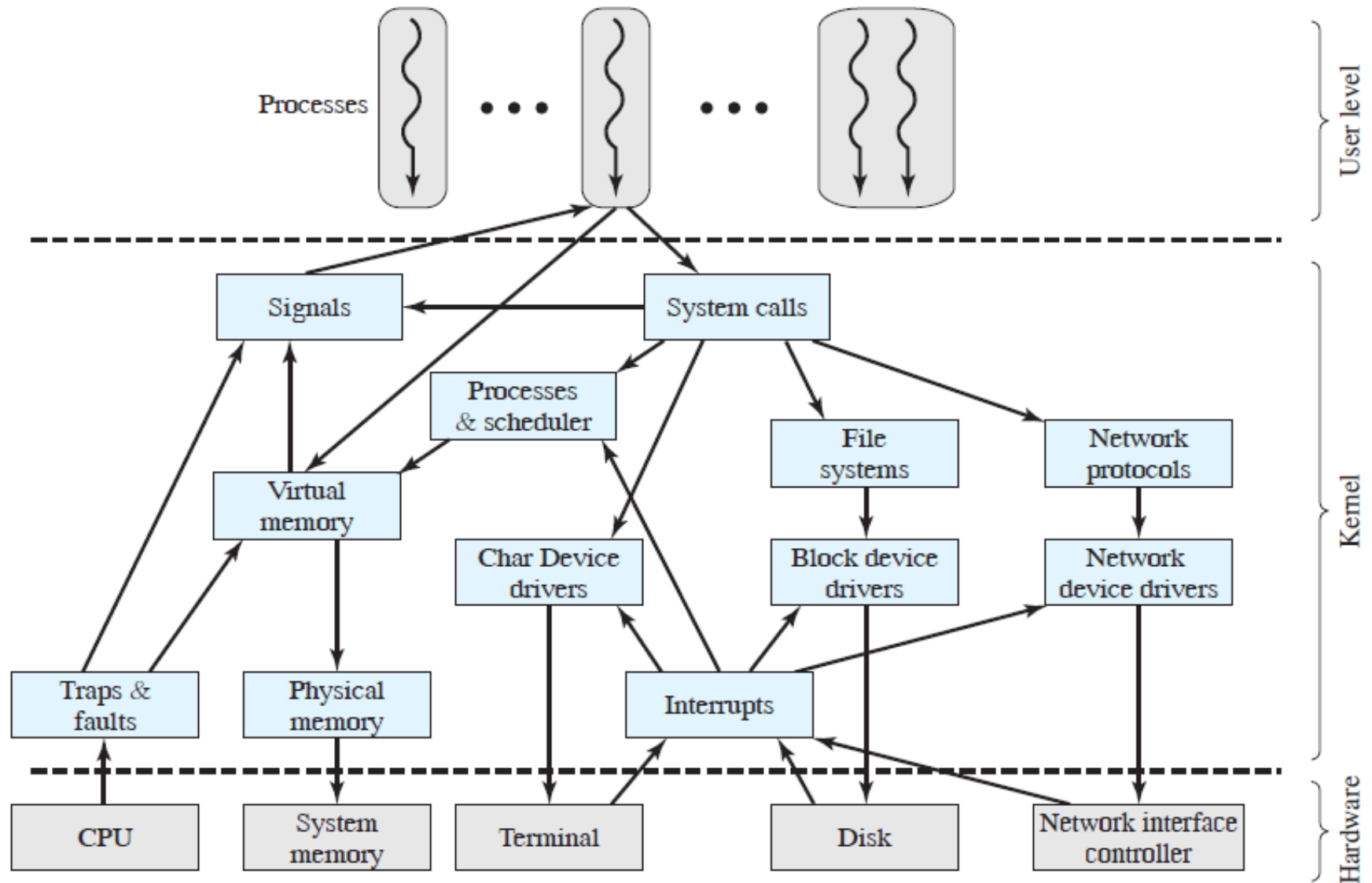
Contoh: - MacOS

- Windows NT dan turunannya (makin mendekati monolithic)

Ringkasan



Linux Kernel Components

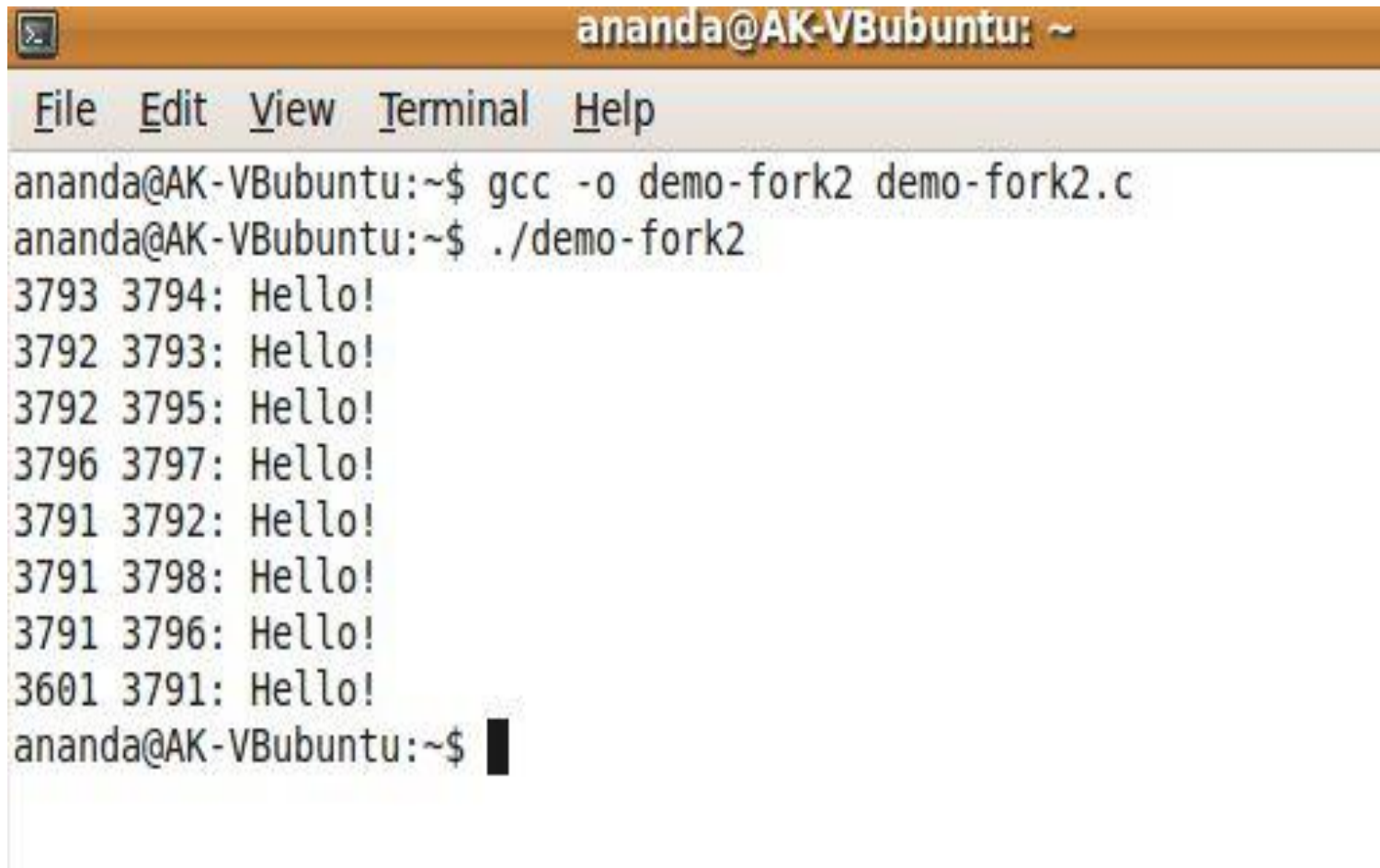


Contoh:
System Call: fork()

Hirarki Process dengan system call fork()

```
demo-fork2.c
1 #include <unistd.h>
2 #include <stdio.h>
3
4 int main()
5 {
6     fork();
7     fork();
8     fork();
9     printf("%d %d: Hello!\n",getppid(),getpid());
10 }
```

Hirarki Process dengan system call fork()



```
ananda@AK-VBubuntu: ~  
File Edit View Terminal Help  
ananda@AK-VBubuntu:~$ gcc -o demo-fork2 demo-fork2.c  
ananda@AK-VBubuntu:~$ ./demo-fork2  
3793 3794: Hello!  
3792 3793: Hello!  
3792 3795: Hello!  
3796 3797: Hello!  
3791 3792: Hello!  
3791 3798: Hello!  
3791 3796: Hello!  
3601 3791: Hello!  
ananda@AK-VBubuntu:~$
```

Akhir Kuliah Minggu 2

Terima kasih atas perhatiannya!